

Exercise 2: E-commerce Platform Search Function

Objective:

To implement Linear Search and Binary Search algorithms for searching products in an e-commerce platform and compare their performance using time complexity analysis.

Problem Statement:

An e-commerce platform contains a large number of products. Customers search for products by name. The search functionality should be efficient and optimized.

The task is to:

- Create a Product class.
- Store product details in an array.
- Implement Linear Search.
- Implement Binary Search.
- Compare both algorithms based on time complexity.

Theory:

Linear Search

Linear Search checks each element one by one until the required product is found.

Advantages

- Simple to implement.
- Works on unsorted data.

Disadvantages

- Slow for large datasets.

Binary Search

Binary Search divides the sorted array into two halves repeatedly until the required product is found.

Advantages

- Very fast.

- Efficient for large sorted datasets.

Disadvantages

- Works only on sorted data.

Algorithm

Linear Search Algorithm

1. Start from the first product.
2. Compare the product name with the search key.
3. If matched, return the product.
4. Otherwise continue searching.
5. If no product matches, return "Product Not Found".

Binary Search Algorithm

1. Set Left = 0.
2. Set Right = Last Index.
3. Find the middle element.
4. Compare the middle element with the search key.
5. If equal, return the product.
6. If search key is greater, search the right half.
7. Otherwise search the left half.
8. Repeat until found or the search space becomes empty.

Program:

using System;

public class Product

{

 public int ProductId;

 public string ProductName;

 public string Category;

 public Product(int id, string name, string category)

```
{
    ProductId = id;
    ProductName = name;
    Category = category;
}
}
public class Program
{
    // Linear Search
    static Product LinearSearch(Product[] products, string searchName)
    {
        foreach (Product product in products)
        {
            if (product.ProductName.Equals(searchName,
StringComparison.OrdinalIgnoreCase))
            {
                return product;
            }
        }
        return null;
    }
    // Binary Search
    static Product BinarySearch(Product[] products, string searchName)
    {
        int left = 0;
        int right = products.Length - 1;
        while (left <= right)
        {
```

```
        int mid = (left + right) / 2;

        int compare = string.Compare(products[mid].ProductName,
searchName, true);

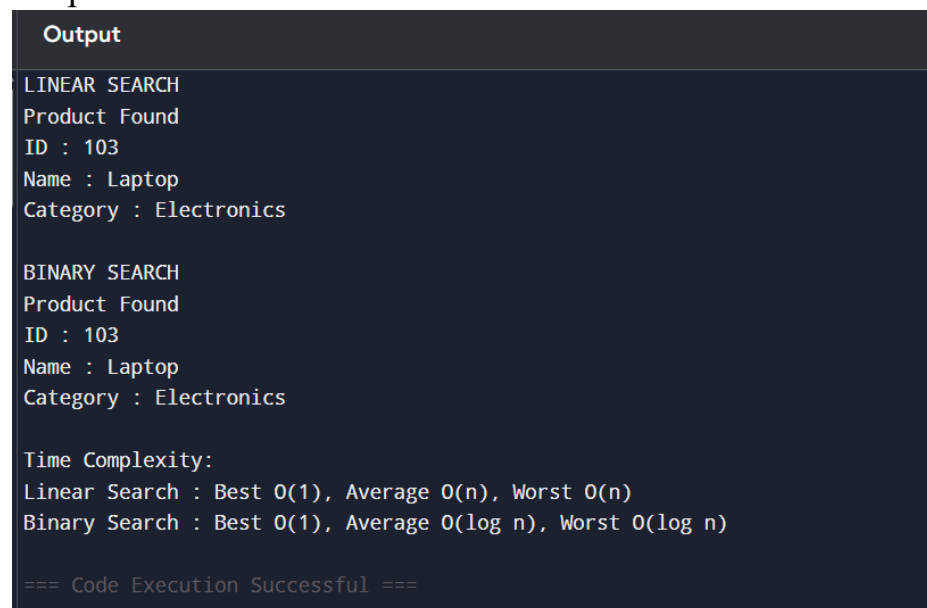
        if (compare == 0)
        {
            return products[mid];
        }
        else if (compare < 0)
        {
            left = mid + 1;
        }
        else
        {
            right = mid - 1;
        }
    }
    return null;
}

public static void Main(string[] args)
{
    Product[] products =
    {
        new Product(101, "Camera", "Electronics"),
        new Product(102, "Headphones", "Electronics"),
        new Product(103, "Laptop", "Electronics"),
        new Product(104, "Mobile", "Electronics"),
        new Product(105, "Shoes", "Fashion")
    };
}
```

```
Console.WriteLine("LINEAR SEARCH");
Product linearResult = LinearSearch(products, "Laptop");
if (linearResult != null)
{
    Console.WriteLine("Product Found");
    Console.WriteLine("ID : " + linearResult.ProductId);
    Console.WriteLine("Name : " + linearResult.ProductName);
    Console.WriteLine("Category : " + linearResult.Category);
}
else
{
    Console.WriteLine("Product Not Found");
}
Console.WriteLine();
Console.WriteLine("BINARY SEARCH");
Product binaryResult = BinarySearch(products, "Laptop");
if (binaryResult != null)
{
    Console.WriteLine("Product Found");
    Console.WriteLine("ID : " + binaryResult.ProductId);
    Console.WriteLine("Name : " + binaryResult.ProductName);
    Console.WriteLine("Category : " + binaryResult.Category);
}
else
{
    Console.WriteLine("Product Not Found");
}
```

```
}  
Console.WriteLine();  
Console.WriteLine("Time Complexity:");  
Console.WriteLine("Linear Search : Best O(1), Average O(n), Worst  
O(n)");  
Console.WriteLine("Binary Search : Best O(1), Average O(log n), Worst  
O(log n)") }}
```

Output:



```
Output  
LINEAR SEARCH  
Product Found  
ID : 103  
Name : Laptop  
Category : Electronics  
  
BINARY SEARCH  
Product Found  
ID : 103  
Name : Laptop  
Category : Electronics  
  
Time Complexity:  
Linear Search : Best O(1), Average O(n), Worst O(n)  
Binary Search : Best O(1), Average O(log n), Worst O(log n)  
  
=== Code Execution Successful ===
```

Conclusion

This exercise demonstrates the implementation of Linear Search and Binary Search for searching products in an e-commerce platform. Linear Search is suitable for small or unsorted datasets, whereas Binary Search provides significantly better performance for sorted datasets because it reduces the search space by half during each iteration.